

CS342L – Advanced Database Management Systems Lab

ADVANCED SQL 1: ALL CLAUSES



Ghulam Ishaq Khan Institute of Engineering Sciences



Objective

The objective of this session is to master every clause of the SQL SELECT statement in PostgreSQL — from SELECT and WHERE through JOINS, GROUP BY, subqueries, and set operators — by querying the dvdrental database migrated in Lab 1.

Instructions

- Open the handout/lab manual in front of a computer in the lab during the session, with the dvdrental database from Lab 1 already restored and available.
- Practice each new command by completing the examples and exercises.
- Turn in the answers for all the exercise problems as your lab report.
- When answering problems, indicate the commands you entered and the output displayed.

Contents

The dvdrental Database (Recap).....	4
The Anatomy of a SELECT Query	4
Clause Order vs. Logical Order of Execution.....	4
The SELECT Clause.....	5
Column Aliases and Expressions	5
The DISTINCT Clause	5
The FROM Clause and Table Aliases	5
The WHERE Clause	6
Comparison and Logical Operators.....	6
BETWEEN and IN	6
Pattern Matching with LIKE	6
Checking for NULL.....	6
The ORDER BY Clause.....	7
The LIMIT and OFFSET Clauses	7
The GROUP BY Clause	7
The HAVING Clause	7
JOIN Clauses	8
INNER JOIN	8
LEFT and RIGHT (OUTER) JOIN	8
FULL (OUTER) JOIN.....	8
CROSS JOIN	9
SELF JOIN.....	9
The USING Shortcut	9
Set Operators: UNION, INTERSECT, EXCEPT	9
Subqueries	10
Scalar and IN Subqueries	10
EXISTS, ANY, and ALL	11
Correlated Subqueries	11
The CASE Expression	11
Putting It All Together.....	12
Exercise	12

The dvdrental Database (Recap)

This lab continues to use the dvdrental database restored in Lab 1. It builds on film, actor, film_actor, category, and film_category (the catalogue side); customer, staff, address, city, country, and language (the people and reference side); and store, inventory, rental, and payment (the business-transaction side). If your dvdrental database is not currently restored, repeat the restore steps from Lab 1 before continuing.

Roughly speaking, a customer belongs to a store and rents an inventory copy of a film; a film belongs to one or more categories through film_category and is offered in a language; a rental generates a payment; and every address links to a city, which links to a country. Keeping this shape in mind makes the JOIN examples later in this lab much easier to read.

The Anatomy of a SELECT Query

A full SELECT statement can combine every clause covered in this lab. PostgreSQL accepts them in a fixed written order, but evaluates them in a different logical order — understanding both is what separates an “advanced” SQL user from a beginner.

```
SELECT    column_list
FROM      table_list
JOIN      other_table ON condition
WHERE     row_filter
GROUP BY  grouping_columns
HAVING    group_filter
ORDER BY  sort_columns
LIMIT    row_count OFFSET skip_count;
```

Code language: SQL (Structured Query Language) (sql)

Clause Order vs. Logical Order of Execution

This is the order you type the clauses in:

1. SELECT / DISTINCT
2. FROM / JOIN
3. WHERE
4. GROUP BY
5. HAVING
6. ORDER BY
7. LIMIT / OFFSET

This is the order PostgreSQL actually evaluates them in:

1. FROM / JOIN — build the working row set from the named tables.
2. WHERE — discard rows that don’t match the row filter.
3. GROUP BY — collapse the remaining rows into groups.
4. HAVING — discard groups that don’t match the group filter.
5. SELECT — compute the output columns and aliases.
6. DISTINCT — remove duplicate output rows.

7. ORDER BY — sort the output.
8. LIMIT / OFFSET — return only the requested slice of rows.

Note: Because WHERE runs before SELECT, you cannot reference a SELECT column alias inside WHERE. Because HAVING runs after GROUP BY, it can filter on aggregates like COUNT(*) or SUM(...) that WHERE cannot see.

The SELECT Clause

SELECT lists which columns — or computed expressions — appear in the result:

```
SELECT first_name, last_name, email
FROM customer;
```

Code language: SQL (Structured Query Language) (sql)

Column Aliases and Expressions

Use AS to rename a column in the output, and build expressions directly in the column list. AS is optional but recommended for readability.

```
SELECT
    first_name || ' ' || last_name AS full_name
FROM customer;
```

Code language: SQL (Structured Query Language) (sql)

```
SELECT title, rental_rate, rental_rate * 1.10 AS rate_with_tax
FROM film;
```

Code language: SQL (Structured Query Language) (sql)

The DISTINCT Clause

DISTINCT removes duplicate rows from the result set after SELECT has computed the output columns.

```
SELECT DISTINCT rating
FROM film
ORDER BY rating;
```

Code language: SQL (Structured Query Language) (sql)

DISTINCT applies to the whole row, so adding a column can change what counts as a duplicate:

```
SELECT DISTINCT rating, length
FROM film
ORDER BY rating, length;
```

Code language: SQL (Structured Query Language) (sql)

The FROM Clause and Table Aliases

FROM names the table (or tables) a query reads from. Giving a table a short alias makes multi-table queries far easier to write and read — this becomes essential once JOINS are introduced.

```
SELECT f.title, f.rental_rate
FROM film AS f
ORDER BY f.title;
```

Code language: SQL (Structured Query Language) (sql)

Note: The AS keyword before a table alias is optional — “FROM film f” works identically to “FROM film AS f”.

The WHERE Clause

WHERE filters individual rows before any grouping happens.

Comparison and Logical Operators

```
SELECT title, rental_rate
FROM film
WHERE rental_rate > 4.99;
```

Code language: SQL (Structured Query Language) (sql)

```
SELECT title, rating, rental_duration
FROM film
WHERE rating = 'PG' AND rental_duration > 5;
```

Code language: SQL (Structured Query Language) (sql)

BETWEEN and IN

```
SELECT title, length
FROM film
WHERE length BETWEEN 60 AND 90
ORDER BY length;
```

Code language: SQL (Structured Query Language) (sql)

```
SELECT title, rating
FROM film
WHERE rating IN ('G', 'PG')
ORDER BY title;
```

Code language: SQL (Structured Query Language) (sql)

Pattern Matching with LIKE

LIKE matches text patterns: % stands for any sequence of characters, _ stands for exactly one character.

```
SELECT title
FROM film
WHERE title LIKE 'A%'
ORDER BY title;
```

Code language: SQL (Structured Query Language) (sql)

Checking for NULL

A rental that has not yet been returned has a NULL `return_date`. NULL must be tested with `IS NULL` / `IS NOT NULL` — never with `=` or `<>` — because NULL never equals anything, including itself.

```
SELECT rental_id, customer_id, rental_date
FROM rental
WHERE return_date IS NULL
ORDER BY rental_date;
```

Code language: SQL (Structured Query Language) (sql)

The ORDER BY Clause

ORDER BY sorts the final result. You can sort by one or more columns, each independently ascending (ASC, the default) or descending (DESC):

```
SELECT title, rating, rental_rate
FROM film
ORDER BY rental_rate DESC, title ASC;
```

Code language: SQL (Structured Query Language) (sql)

The LIMIT and OFFSET Clauses

LIMIT caps the number of rows returned; OFFSET skips a number of rows first — useful for paging through results.

```
SELECT title
FROM film
ORDER BY title
LIMIT 10 OFFSET 20; -- the third page of 10 results
```

Code language: SQL (Structured Query Language) (sql)

The GROUP BY Clause

GROUP BY collapses rows that share the same value(s) into a single row per group, typically paired with an aggregate function such as COUNT, SUM, AVG, MIN, or MAX.

```
SELECT rating, COUNT(*) AS film_count
FROM film
GROUP BY rating
ORDER BY film_count DESC;
```

Code language: SQL (Structured Query Language) (sql)

The HAVING Clause

HAVING filters groups after GROUP BY has run — it is to groups what WHERE is to individual rows, and it is the only clause that can filter on an aggregate result.

```
SELECT customer_id, COUNT(*) AS rental_count
FROM rental
GROUP BY customer_id
HAVING COUNT(*) > 30
ORDER BY rental_count DESC;
```

Code language: SQL (Structured Query Language) (sql)

Note: WHERE cannot use COUNT(*) or SUM(...) because it runs before grouping happens; use HAVING for that instead.

JOIN Clauses

A JOIN combines rows from two tables based on a matching condition. dvdrental is fully normalized, so almost every useful query needs at least one JOIN.

INNER JOIN

INNER JOIN (or just JOIN) keeps only rows that have a match in both tables:

```
SELECT r.rental_id, c.first_name, c.last_name, r.rental_date
FROM rental r
JOIN customer c ON c.customer_id = r.customer_id
ORDER BY r.rental_date
LIMIT 10;
```

Code language: SQL (Structured Query Language) (sql)

LEFT and RIGHT (OUTER) JOIN

LEFT JOIN keeps every row from the left table, filling in NULLs for any right-table columns that have no match. This is the standard way to find rows in one table that have nothing matching them in another — for example, films that are not currently stocked in inventory:

```
SELECT f.film_id, f.title
FROM film f
LEFT JOIN inventory i ON i.film_id = f.film_id
WHERE i.inventory_id IS NULL
ORDER BY f.title;
```

Code language: SQL (Structured Query Language) (sql)

RIGHT JOIN is the mirror image — it keeps every row from the right table instead:

```
SELECT f.title, i.inventory_id
FROM inventory i
RIGHT JOIN film f ON f.film_id = i.film_id
WHERE i.inventory_id IS NULL
ORDER BY f.title;
```

Code language: SQL (Structured Query Language) (sql)

Note: A RIGHT JOIN can always be rewritten as a LEFT JOIN by swapping the table order — the two queries above return the same rows.

FULL (OUTER) JOIN

FULL JOIN keeps every row from both tables, matching where possible and filling in NULLs on whichever side has no match:

```
SELECT f.title, i.inventory_id
FROM film f
FULL JOIN inventory i ON i.film_id = f.film_id
WHERE i.inventory_id IS NULL;
```

Code language: SQL (Structured Query Language) (sql)

Note: Because every inventory row is required to reference a real film, this particular FULL JOIN returns the same rows as the LEFT JOIN example above. FULL JOIN only differs from LEFT JOIN when the right-hand table can also contain unmatched rows.

CROSS JOIN

CROSS JOIN returns the Cartesian product of two tables — every row of the first paired with every row of the second, with no ON condition:

```
SELECT s.store_id, c.name AS category_name
FROM store s
CROSS JOIN category c
ORDER BY s.store_id, c.name;
```

Code language: SQL (Structured Query Language) (sql)

Note: With 2 stores and 16 categories, this CROSS JOIN returns exactly 32 rows (2×16).

SELF JOIN

A self join joins a table to itself — useful for comparing rows within the same table. This finds pairs of different films that share exactly the same length:

```
SELECT f1.title AS film_a, f2.title AS film_b, f1.length
FROM film f1
JOIN film f2 ON f1.length = f2.length AND f1.film_id < f2.film_id
ORDER BY f1.length
LIMIT 10;
```

Code language: SQL (Structured Query Language) (sql)

Note: $f1.film_id < f2.film_id$ avoids matching a film with itself and avoids listing every pair twice.

The USING Shortcut

When the join columns have the exact same name in both tables, USING is a shorter alternative to ON:

```
SELECT title, inventory_id
FROM film
JOIN inventory USING (film_id);
```

Code language: SQL (Structured Query Language) (sql)

Set Operators: UNION, INTERSECT, EXCEPT

Set operators combine the results of two SELECT statements that return the same number of columns with compatible types. UNION removes duplicate rows by default; UNION ALL keeps them.

```
SELECT city FROM city WHERE city LIKE 'A%'
UNION
SELECT city FROM city WHERE city LIKE '%a'
ORDER BY city;
```

Code language: SQL (Structured Query Language) (sql)

Replacing UNION with UNION ALL above would keep both copies of any city name matched by both conditions.

INTERSECT returns only the rows that appear in both result sets — for example, cities where dvdrental has both a customer and a staff member living:

```
SELECT ci.city
FROM city ci
JOIN address a ON a.city_id = ci.city_id
JOIN customer c ON c.address_id = a.address_id
INTERSECT
SELECT ci.city
FROM city ci
JOIN address a ON a.city_id = ci.city_id
JOIN staff s ON s.address_id = a.address_id;
```

Code language: SQL (Structured Query Language) (sql)

EXCEPT returns rows from the first query that do not appear in the second — for example, films customer_id = 1 has rented that customer_id = 2 has not:

```
SELECT i.film_id
FROM rental r
JOIN inventory i ON i.inventory_id = r.inventory_id
WHERE r.customer_id = 1
EXCEPT
SELECT i.film_id
FROM rental r
JOIN inventory i ON i.inventory_id = r.inventory_id
WHERE r.customer_id = 2;
```

Code language: SQL (Structured Query Language) (sql)

Subqueries

A subquery is a SELECT nested inside another statement. It can return a single value (scalar), a list of values, or a whole table, depending on where it is used.

Scalar and IN Subqueries

A scalar subquery returns exactly one value and can be used anywhere a single value is expected:

```
SELECT title, rental_rate
FROM film
WHERE rental_rate = (SELECT MAX(rental_rate) FROM film);
```

Code language: SQL (Structured Query Language) (sql)

An IN subquery returns a list of values to match against:

```
SELECT DISTINCT first_name, last_name
FROM customer
WHERE customer_id IN (
    SELECT customer_id FROM payment WHERE amount > 10
);
```

Code language: SQL (Structured Query Language) (sql)

EXISTS, ANY, and ALL

EXISTS checks only whether a subquery returns any rows at all — it never looks at the actual values, which often makes it faster than IN:

```
SELECT c.first_name, c.last_name
FROM customer c
WHERE EXISTS (
    SELECT 1 FROM rental r
    WHERE r.customer_id = c.customer_id AND r.return_date IS NULL
);
```

Code language: SQL (Structured Query Language) (sql)

ANY is true if the comparison holds for at least one row the subquery returns; ALL requires it to hold for every row:

```
-- true for almost every film: only needs to beat the cheapest G-rated price
SELECT title, rental_rate FROM film
WHERE rental_rate >= ANY (SELECT rental_rate FROM film WHERE rating = 'G');

-- true only for films priced at or above the priciest G-rated price
SELECT title, rental_rate FROM film
WHERE rental_rate >= ALL (SELECT rental_rate FROM film WHERE rating = 'G');
```

Code language: SQL (Structured Query Language) (sql)

Correlated Subqueries

A correlated subquery references a column from the outer query, so it is re-evaluated once per outer row rather than just once. This finds each customer's total rental count without a GROUP BY:

```
SELECT
    c.customer_id, c.first_name, c.last_name,
    (SELECT COUNT(*) FROM rental r WHERE r.customer_id = c.customer_id) AS rental_count
FROM customer c
ORDER BY rental_count DESC
LIMIT 5;
```

Code language: SQL (Structured Query Language) (sql)

The CASE Expression

CASE lets a query branch on a condition and return different values — SQL's equivalent of if/else:

```

SELECT
    title,
    length,
    CASE
        WHEN length < 60 THEN 'Short'
        WHEN length BETWEEN 60 AND 120 THEN 'Medium'
        ELSE 'Long'
    END AS length_category
FROM film
ORDER BY length;

```

Code language: SQL (Structured Query Language) (sql)

Putting It All Together

A realistic report query typically combines several clauses at once. This finds the top 5 highest-grossing film categories, using JOIN, GROUP BY, HAVING, ORDER BY, and LIMIT together:

```

SELECT cat.name AS category, SUM(p.amount) AS revenue
FROM payment p
JOIN rental r    ON r.rental_id = p.rental_id
JOIN inventory i  ON i.inventory_id = r.inventory_id
JOIN film f      ON f.film_id = i.film_id
JOIN film_category fc ON fc.film_id = f.film_id
JOIN category cat ON cat.category_id = fc.category_id
GROUP BY cat.name
HAVING SUM(p.amount) > 1000
ORDER BY revenue DESC
LIMIT 5;

```

Code language: SQL (Structured Query Language) (sql)

Reading it in logical order: PostgreSQL builds the joined row set, groups it by category, keeps only categories whose total revenue exceeds \$1000, computes the SUM for the SELECT list, sorts by revenue, and returns the top 5.

Exercise

1. Write a query using DISTINCT to list every film rating that appears in the film table.
2. Write a query using WHERE and LIKE to find every customer whose email ends in @sakilacustomer.org.
3. Write a query using ORDER BY, LIMIT, and OFFSET to return the second page of 10 films, sorted alphabetically by title.
4. Write a query using GROUP BY and HAVING to find every film category that contains more than 60 films.
5. Write a query using a LEFT JOIN to find every film that has never been assigned to a category.
6. Write a query using a subquery to find every customer whose total payments are above the average total payment across all customers.
7. Write a query using EXCEPT to find every actor's first name that is not also used as a customer first name.
8. Write a query using CASE together with GROUP BY to count how many films fall into each rental_rate tier: 'Budget' (under 2.99), 'Standard' (2.99 up to but not including 4.99), or 'Premium' (4.99 and above).